Design System Handoff — Flutter

Oct 20, 2026 · @Mehul Gohil

Build the community app UI from the Figma file and the generated theme files; every colour, spacing, radius and text style in code comes from those tokens, never a typed value.

What you get

Two things: the Mobile Design System Figma file (open it in Dev Mode) and a `lib/theme/` folder generated from its variables.

Item	What it holds	Use it for
Figma › Foundations	Every colour token in Light and Dark, primitives, 24 text styles, spacing and radius scales	Looking up a token's value and name
Figma › Rules	11 numbered usage rules	What is mandatory vs recommended
Figma › Components	11 components with variants, plus 16 icons; each description names its Flutter widget	Building widgets; Light/Dark phone previews show real usage
`app_colors.dart`	`AppColors` — 33 colour tokens, Light + Dark, read with `context.colors`	All colours
`app_spacing.dart` / `app_radius.dart`	`AppSpacing.s0–s64`, `AppRadius.r0–r16` + `rFull`	Padding, gaps, corners
`app_typography.dart`	`AppTypography.h1 … buttonXs` (24 styles, no colour)	All text
`app_shadows.dart`	`AppShadows.sm` / `lg` / `sheet`	Raised tabs, dialogs, sheets
`app_theme.dart`	`AppTheme.light()` / `dark()` — Material `ThemeData`	`MaterialApp(theme:, darkTheme:)`

Names map 1:1 from Figma: `text/secondary` → `context.colors.textSecondary`, `space/16` → `AppSpacing.s16`, `Body/Md Strong` → `AppTypography.bodyMdStrong`.

Setup

Requires Flutter 3.22 or newer (Dart 3). The folder's `README.md` has the full snippets.

1. Copy `lib/theme/` into the app and import `theme/theme.dart` (it exports every file).
2. Bundle the fonts as assets — do not download them at runtime, so they work offline and on iOS:
   - Roboto 400 / 600 / 700 (English)
   - Noto Sans Gujarati 400 / 600 / 700
   - Noto Sans Devanagari 400 / 600 / 700
3. Keep the pubspec `family:` names exactly `Roboto`, `Noto Sans Gujarati`, `Noto Sans Devanagari`; the text styles fall back to the Noto fonts automatically, so mixed Gujarati and English text needs no per-word font.
4. Add `material_symbols_icons` and use the Rounded set at weight 400, e.g. `Symbols.search_rounded`. Do not use Flutter's built-in `Icons.*_rounded` — that older set looks different.

5. Wire the theme: `MaterialApp(theme: AppTheme.light(), darkTheme: AppTheme.dark(), themeMode: ThemeMode.system)`.
6. In custom widgets read tokens directly: `context.colors.bgSurfaceRaised`, `AppSpacing.s16`, `AppTypography.bodyMd.copyWith(color: context.colors.textSecondary)`.

Components

Build one reusable widget per Figma component; its Figma properties become the widget's parameters.

Figma component	Flutter widget	Parameters / notes
Button	`AppButton`	`variant`: Primary, Outline, Neutral, Text, Text Neutral · `size`: Xs 24, Sm 32, Md 40, Lg 48 (default) · `label`, `leadingIcon`, `trailingIcon` · states: pressed, disabled
Input	`AppTextField`	`style`: Label above (default) or Material · states: empty, filled, focused, error, success, disabled · `label` (optional), `hint`, `helperText`, `errorText`, `prefix` (e.g. +91), leading/trailing icon
Status Badge	`StatusBadge`	`tone`: Neutral, Info, Success, Warning, Error, Pending, Progress · `label` · `icon` off by default · read-only, not tappable
Dialog	`AppDialog` via `showDialog`	`icon`: none, info, success, warning, error · `title`, `body` · primary action (Md) + optional secondary (Text Neutral Md), right-aligned · width 328
Bottom Sheet	`AppBottomSheet` via `showModalBottomSheet`	`type`: custom, single select, multi select · `title`, close button, optional search · custom takes any `child` + optional actions
Tabs / Tab	`TabBar` (Underline) · `SegmentedButton` (Segmented)	2–4 equal-width tabs · Underline full width with 2px indicator · Segmented fully rounded, inset 16
Option + Radio / Checkbox	`RadioListTile` / `CheckboxListTile` styled to tokens	48px rows · selected label switches to Body/Md Strong + `textPrimary`
Icons	`Icon(Symbols.<name>_rounded)`	24px grid; colour from `iconDefault` / `iconSubtle` or the parent component

Behaviour not visible in Figma

A static design cannot show these; they are part of the spec.

- **Tap targets:** every tappable element keeps a 48 × 48 touch area, including Xs/Sm/Md buttons and the close icon. `AppTheme` sets `MaterialTapTargetSize.padded`; custom widgets add hit padding.
- **Dark mode primary:** filled primary buttons are `#3FA5F9` with **dark** text (`textOnPrimary`), not white. Do not hard-code white on primary.
- **Theme switching:** follow the system setting (`ThemeMode.system`); never pick colours by checking brightness — read `context.colors`.
- **Scrim:** dialogs and bottom sheets use `context.colors.bgScrim` as `barrierColor` (50% in Light, 70% in Dark). `AppTheme` already sets it for bottom sheets; pass it to `showDialog`.
- **Single-select sheet:** no buttons; tapping an option selects it and closes the sheet.
- **Multi-select sheet:** Apply is disabled and Clear is hidden until at least one option is selected; Clear empties the selection.

- **Bottom sheets:** add the device safe-area inset at the bottom, scroll long lists inside the sheet, cap height at about 90% of the screen (`isScrollControlled: true`).
- **Material-style inputs:** only on the screen background — the floating-label cut-out is painted with `bgScreen`.
- **Text:** long single-line labels truncate with an ellipsis (buttons, tabs, option rows); body text wraps. Avoid 10px Button/Xs for Gujarati labels.
- **Status:** never rely on colour alone — every badge has a text label.

Rules and changes

The Rules page in Figma is the reference; these four matter most in code:

- Colours only from `context.colors` — no hex values, no `Colors.grey`, no primitives.
- Sizes, spacing, radius and font sizes are even numbers from the scales; no one-off values.
- One Primary button per screen; secondary actions use Outline, Neutral or Text.
- Text keeps ≥ 4.5:1 contrast and icons/borders ≥ 3:1 — the tokens already meet this in both modes, so changing a token pairing needs a design check.

How a design change reaches the app:

1. The design is changed in Figma (variables, text or effect styles).
2. The theme files are regenerated from Figma and sent over — they are marked `GENERATED`; do not edit them by hand.
3. Replace `lib/theme/` with the new files; widgets built on tokens update with no other code changes.

A missing token or component is a design question, not a code workaround: raise it with Mag before adding a one-off value.